Tutorial 4: Hardware–Software Interface and Memory Layout

ES 336 – Computer Organization and Architecture

Indian Institute of Technology Gandhinagar

Problem 1: Compiler Stack

Generate the intermediate files to verify their order and contents.

Step 1: .c → .i

Despite no headers included in .c, why does the preprocessor output still show some headers injected?

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P1_Cstack$ vim p1.c
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P1_Cstack$ cat p1.c
int add(int a, int b) {
return a + b;
}
int main() {
return add(2, 3);
}
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P1_Cstack$ gcc -E p1.c -o p1.i
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P1_Cstack$ cat p1.i
# 1 "p1.c"
# 1 "<built-in>"
# 1 "<command-line>"
# 31 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 32 "<command-line>" 2
# 1 "p1.c"
int add(int a, int b) {
return a + b;
}
int main() {
return add(2, 3);
}
```

These headers, added by GCC, only define internal macros and do not affect program logic or symbol generation. Used so that the compiler knows how to interpret the C standard.

Step 2: .i → .s Compilation

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P1_Cstack$ gcc -S p1.i -o p1.s
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P1_Cstack$ cat p1.s
        .file   "p1.c"
        .text
        .globl  add
        .type   add, @function
add:
.LFB0:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        movl    %edi, -4(%rbp)
        movl    %esi, -8(%rbp)
        movl    -4(%rbp), %edx
        movl    -8(%rbp), %eax
        addl    %edx, %eax
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE0:
        .size   add, .-add
        .globl  main
        .type   main, @function
main:
.LFB1:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
```

Figure 1: Assembly code generated

Step 3: .s → .o Assembly

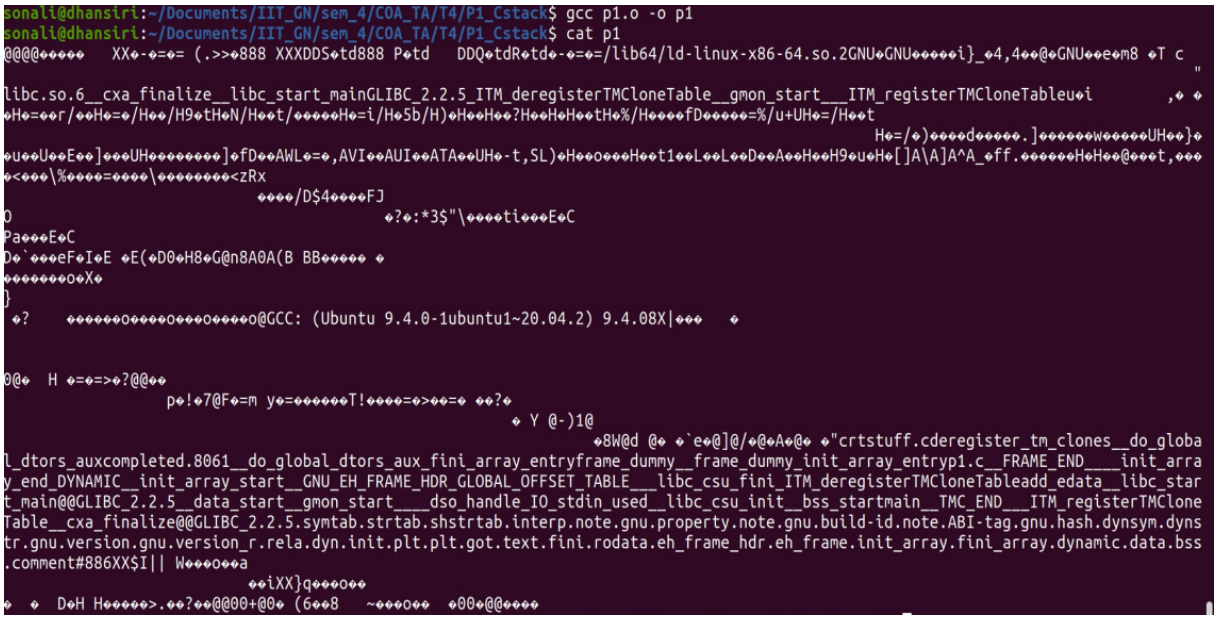
```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P1_Cstack$ gcc -c p1.s -o p1.o
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P1_Cstack$ cat p1.o
ELF>◆@@
◆◆UH◆◆]◆◆u◆◆U◆◆E◆◆]◆◆◆UH◆◆◆◆]◆GCC: (Ubuntu 9.4.0-1ubuntu1~20.04.2) 9.4.0GNU◆zRx
O                                                                    E◆C
PE◆C
◆◆
p1.caddmain+    ◆◆◆◆◆◆◆◆ @.symtab.strtab.shstrtab.rela.text.data.bss.comment.note.GNU-stack.note.gnu.property.rela.eh_frame @10
&qq10q,:◆J◆b◆]@H0

        xlsonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P1_Cstack$
```

Convert assembly code to machine code. Non-comprehensible for humans!!

Step 4: .o → executable or binary Linking

If you have multiple machine-level codes, link them together. Again Non-comprehensible!!



- **test.c**: High-level C source code written by the programmer.
- **test.i**: Source code with preprocessor directives expanded (macros replaced, header files included).
- **test.s**: Human-readable assembly instructions (mnemonics) specific to the target architecture.
- **test.o**: Relocatable machine code (binary) that has not yet been linked with system libraries.
- **executable**: Final executable machine code with all external library references resolved and linked.

Problem 2: Compiler Stack - Cross compilation

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P2_crossC$ arm-linux-gnueabi-gcc p2.c -o program_arm32
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P2_crossC$ file program_arm32
program_arm32: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.3, BuildID[s
ha1]=be0f651ad9c832e71f850dc19bb48120a542d5f8, for GNU/Linux 3.2.0, not stripped
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P2_crossC$ aarch64-linux-gnu-gcc program.c -o program_arm64
aarch64-linux-gnu-gcc: error: program.c: No such file or directory
aarch64-linux-gnu-gcc: fatal error: no input files
compilation terminated.
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P2_crossC$ aarch64-linux-gnu-gcc p2.c -o program_arm64
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P2_crossC$ file program_arm64
program_arm64: ELF 64-bit LSB shared object, ARM aarch64, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux-aarch64.so
.1, BuildID[sha1]=9934cd064789525e6f00221ee804c431249fafa4, for GNU/Linux 3.7.0, not stripped
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P2_crossC$ aarch64-linux-gnu-gcc -E p2.c -o p2.i
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P2_crossC$ aarch64-linux-gnu-gcc -S p2.i -o p2.s
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P2_crossC$ aarch64-linux-gnu-gcc -c p2.s -o p2.o
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P2_crossC$ aarch64-linux-gnu-gcc p2.o -o p2_ARM
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P2_crossC$ ./p2_ARM
bash: ./p2_ARM: cannot execute binary file: Exec format error
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P2_crossC$
```

Identify your machine architecture and compile accordingly for any other architecture. Need architecture-specific cross-compilers: sudo apt install gcc-aarch64-linux-gnu
Compare the assembly code for both architectures:

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P1_Cstack$ cat p1.s
        .file   "p1.c"
        .text
        .globl  add
        .type   add, @function
add:
.LFB0:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        movl    %edi, -4(%rbp)
        movl    %esi, -8(%rbp)
        movl    -4(%rbp), %edx
        movl    -8(%rbp), %eax
        addl    %edx, %eax
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
.LFE0:
        .size   add, .-add
        .globl  main
        .type   main, @function
main:
.LFB1:
        .cfi_startproc
        endbr64
        pushq   %rbp
```

Figure 2: Host Machine: x86_64

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P2_crossC$ cat p2.s
        .arch armv8-a
        .file   "p2.c"
        .text
        .align  2
        .global add
        .type   add, %function
add:
.LFB0:
        .cfi_startproc
        sub     sp, sp, #16
        .cfi_def_cfa_offset 16
        str     w0, [sp, 12]
        str     w1, [sp, 8]
        ldr     w1, [sp, 12]
        ldr     w0, [sp, 8]
        add     w0, w1, w0
        add     sp, sp, 16
        .cfi_def_cfa_offset 0
        ret
        .cfi_endproc
.LFE0:
        .size   add, .-add
        .align  2
        .global main
        .type   main, %function
main:
.LFB1:
        .cfi_startproc
        stp     x29, x30, [sp, -16]!
        .cfi_def_cfa_offset 16
```

Figure 3: Cross Compiled for: ARM 64-bit

Problem 3: Compiler Stack - Optimisation

Compare the assembly codes.

1. gcc O0/1/2/3/s -S p3.c -o p3/O0/1/2/3/s.s

```
main:
.LFB1:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        movl    $3, %esi
        movl    $2, %edi
        call    add
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
```

Figure 4: O0 Level

```
main:
.LFB1:
        .cfi_startproc
        endbr64
        movl    $5, %eax
        ret
        .cfi_endproc
```

Figure 5: O1/2/3/s Level

1. gcc O0/1/2/3/s -S p3.c -o p3$_O$0/1/2/3/s.s

```
add:
.LFB0:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        movl    %edi, -4(%rbp)
        movl    %esi, -8(%rbp)
        movl    -4(%rbp), %edx
        movl    -8(%rbp), %eax
        addl    %edx, %eax
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
```

Figure 6: O0/1 Level

```
add:
.LFB0:
        .cfi_startproc
        endbr64
        leal    (%rdi,%rsi), %eax
        ret
        .cfi_endproc
```

Figure 7: O2/3/s Level

Compare the size of executables/binaries generated with optimisation flags:

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P3_opti$ size p3_O0
   text    data     bss     dec     hex filename
   1490     544       8    2042     7fa p3_O0
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P3_opti$ size p3_O1
   text    data     bss     dec     hex filename
   1434     544       8    1986     7c2 p3_O1
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P3_opti$ size p3_O2
   text    data     bss     dec     hex filename
   1450     544       8    2002     7d2 p3_O2
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P3_opti$ size p3_O3
   text    data     bss     dec     hex filename
   1450     544       8    2002     7d2 p3_O3
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P3_opti$ size p3_Os
   text    data     bss     dec     hex filename
   1450     544       8    2002     7d2 p3_Os
```

Compare the size of executables/binaries generated with optimisation flags.

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P3_opti$ size p3_O0
   text    data     bss     dec     hex filename
   1490     544       8    2042     7fa p3_O0
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P3_opti$ size p3_O1
   text    data     bss     dec     hex filename
   1434     544       8    1986     7c2 p3_O1
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P3_opti$ size p3_O2
   text    data     bss     dec     hex filename
   1450     544       8    2002     7d2 p3_O2
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P3_opti$ size p3_O3
   text    data     bss     dec     hex filename
   1450     544       8    2002     7d2 p3_O3
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P3_opti$ size p3_Os
   text    data     bss     dec     hex filename
   1450     544       8    2002     7d2 p3_Os
```

- Only text section changes.
- data and bss constant.
- -O1 gives the smallest binary??

Try optimisations with other operations:

- $\text{res} += (i * 7 + i/3 - i\%5) \wedge (i << 2);$
- Function calls – Explore inlining

- Matrix multiplication
- Nested loops

Problem 4: Binary Analysis

1. **Magic Number**
   - **Hex Values:** `7F 45 4C 46`
   - **ASCII Representation:** `.  E L F`
   - **Purpose:** This "signature" at the very start of the file tells the Operating System loader that this file is an ELF (Executable and Linkable Format) binary, distinguishing it from images, text files, or other binary formats.
2. **Endianness**
   - **System Type: Little Endian** (Standard for x86-64 Intel/AMD processors).
   - **Byte Sequence for `0x12345678`:**

     `Address 0:` `0x78` (Least Significant Byte)
     `Address 1:` `0x56`
     `Address 2:` `0x34`
     `Address 3:` `0x12` (Most Significant Byte)

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P4_magic$ gcc p4.c -o p4
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P4_magic$ hexdump -C -n 16 p4
00000000  7f 45 4c 46 02 01 01 00  00 00 00 00 00 00 00 00  |.ELF............|
00000010
```

- Do all executable/binary files have the same magic number?? Check. Is it architecture-dependent?
- The OS loader reads these bytes first to verify that the file is a valid Executable and Linkable Format (ELF) binary before attempting to allocate memory or execute code.

Problem 5: Memory Layout Analysis

Write a program to inspect the addresses of all major memory segments: Text, Data, BSS, Heap, and Stack.

1. Create Proof Program

Create a file `check_memory.c`. This updated code includes uninitialized variables and constants to show the full memory map.

```c
#include <stdio.h>
#include <stdlib.h>

int global_init = 10;         // Initialized Data Segment
int global_uninit;            // BSS Segment (Uninitialized)
const int read_only = 100;    // Text/RoData Segment (Constants)

int main() {
    static int static_var = 5;   // Initialized Data Segment
    int x = 5;                   // Stack
    int *p = malloc(sizeof(int)); // Heap
    char *str = "Hello";         // Text/RoData (String Literal)

    printf("=== Memory Segments ===\n");
    printf("Text (Function Code):  %p\n", (void*)main);
    printf("Text (String Literal): %p\n", (void*)str);
    printf("Text (Const Variable): %p\n", (void*)&read_only);
    printf("Data (Global Init):    %p\n", (void*)&global_init);
    printf("Data (Static Init):    %p\n", (void*)&static_var);
    printf("BSS (Global Uninit):   %p\n", (void*)&global_uninit);
    printf("Heap (Malloc):         %p\n", (void*)p);
    printf("Stack (Local Var):     %p\n", (void*)&x);

    free(p);
    return 0;
}
```

2. Run and Analyze

```
gcc check_memory.c -o check_memory
./check_memory
```

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P5_mem_layout$ gcc p5.c -o p5
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P5_mem_layout$ nm p5
0000000000004018 B __bss_start
0000000000004018 b completed.8061
                 w __cxa_finalize@@GLIBC_2.2.5
0000000000004000 D __data_start
0000000000004000 W data_start
0000000000001110 t deregister_tm_clones
0000000000001180 t __do_global_dtors_aux
0000000000003da0 d __do_global_dtors_aux_fini_array_entry
0000000000004008 D __dso_handle
0000000000003da8 d _DYNAMIC
0000000000004018 D _edata
0000000000004020 B _end
0000000000001378 T _fini
00000000000011c0 t frame_dummy
0000000000003d98 d __frame_dummy_init_array_entry
000000000000227c r __FRAME_END__
                 U free@@GLIBC_2.2.5
0000000000004010 D global_init
0000000000003f98 d _GLOBAL_OFFSET_TABLE_
000000000000401c B global_uninit
                 w __gmon_start__
0000000000002130 r __GNU_EH_FRAME_HDR
0000000000001000 t _init
0000000000003da0 d __init_array_end
0000000000003d98 d __init_array_start
0000000000002000 R _IO_stdin_used
                 w _ITM_deregisterTMCloneTable
                 w _ITM_registerTMCloneTable
0000000000001370 T __libc_csu_fini
0000000000001300 T __libc_csu_init
```

Variable / Value	Memory Segment	Reasoning
1. `global_init`	**Data**	Global variables with explicit initialization reside in the initialized Data segment.
2. `global_uninit`	**BSS**	Uninitialized globals are automatically zeroed and stored in the BSS (Block Started by Symbol).
3. `read_only_var`	**Text (RoData)**	`const` globals are placed in read-only memory (often .rodata section within Text) to enforce immutability.
4. `static_init`	**Data**	Static variables retain values across calls; initialized ones go to Data.
5. `static_uninit`	**BSS**	Uninitialized static variables are zeroed and stored in BSS.
6. `local_var`	**Stack**	Standard local variables (automatic storage) live on the Stack.
7. Pointer `msg`	**Stack**	The *pointer variable itself* is a local variable stored on the Stack.
8. String `"Hello"`	**Text (RoData)**	String literals are constant, shared, and stored in read-only memory.
9. Pointer `ptr`	**Stack**	The *pointer variable itself* is local (Stack).
10. `malloc` result	**Heap**	Memory requested via `malloc` is always allocated on the Heap.

Problem 6: Linker

In this problem, let

$$\text{REF}(x.i) \rightarrow \text{DEF}(x.k)$$

denote that the linker associates a reference to symbol x in module i with the definition of x in module k.

If there is a **link-time error**, write **ERROR**. If the linker arbitrarily chooses one of

multiple weak definitions, write **UNKNOWN**.

Program A

```
/* foo1.c */
int main() {
    return 0;
}

/* bar1.c */
int main() {
    return 0;
}
```

Questions

1. What happens when these modules are compiled and linked together.

Program B

```
/* foo5.c */
#include <stdio.h>
void f(void);

int x = 15213;
int y = 15212;

int main() {
    f();
    printf("x = 0x%x y = 0x%x\n", x, y);
    return 0;
}

/* bar5.c */
double x;

void f(void) {
    x = -0.0;
}
```

Questions

1. REF(x.1) → DEF(?)
2. REF(x.2) → DEF(?)
3. Does the linker report a multiple-definition error for symbol x?

Program C

```
/* Module 1: m1.c */
extern void foo(void);

int x;

void bar(void) {
    foo();
}
```

```
/* Module 2: m2.c */
static void foo(void) {
}

int x = 42;
```

Questions

1. REF(foo.1) → DEF(?)
2. REF(x.1) → DEF(?)
3. REF(x.2) → DEF(?)

Solution

At compile time, the compiler classifies each global symbol as either *strong* or *weak*, and the assembler records this information in the symbol table of the relocatable object file. Functions and initialized global variables generate strong symbols, while uninitialized global variables generate weak symbols. During linking, Unix linkers resolve multiply defined symbols using the following rules:

- **Rule 1:** Multiple strong symbol definitions are not allowed and result in a link-time error.

- **Rule 2:** If a strong symbol and one or more weak symbols exist, the linker chooses the strong symbol.
- **Rule 3:** If multiple weak symbols exist, the linker arbitrarily chooses one of them.

Program A Analysis Both modules define the global function `main`, which is a strong symbol. According to Rule 1 of symbol resolution, multiple strong definitions of the same symbol are not allowed. Consequently, the linker reports a link-time error.

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P9$ gcc a_m1.c a_m2.c
/usr/bin/ld: /tmp/ccpynQp6.o: in function `main':
a_m2.c:(.text+0x0): multiple definition of `main'; /tmp/ccBobIY3.o:a_m1.c:(.text+0x0): first defined here
collect2: error: ld returned 1 exit status
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P9$
```

Figure 8: Linker- Part-A.

Program B Analysis

- In Module 1 (`foo5.c`), `x` is an initialized global variable. Therefore, it defines a *strong* global symbol that is visible to the linker.
- In Module 2 (`bar5.c`), `x` is declared as a global variable of type `double` without an initializer. This also introduces a global symbol named `x`, which is treated as a *weak* symbol by the linker.
- On an IA32/Linux machine, doubles are 8 bytes and ints are 4 bytes. Thus, the assignment x = -0.0 in line 6 of bar5.c will overwrite the memory locations for x and y (lines 5 and 6 in foo5.c) with the double-precision floating-point representation of negative zero.

Answers

- (a) REF($x.1$) → DEF($x.1$)
- (b) REF($x.2$) → DEF($x.1$)
- (c) No. The linker does *not* report a multiple-definition error for symbol `x`, because one definition is strong and the other is weak.

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P5_linker$ gcc -c b_m1.c -o bm1.o
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P5_linker$ gcc -c b_m2.c -o bm2.o
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P5_linker$ nm -g b_m1.o
                 U f
                 U _GLOBAL_OFFSET_TABLE_
0000000000000000 T main
                 U printf
0000000000000000 D x
0000000000000004 D y
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P5_linker$ nm -g b_m2.o
0000000000000000 T f
                 U x
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P5_linker$ gcc b_m1.o b_m2.o -o programB
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P5_linker$ nm -g programB | grep x
                 w __cxa_finalize@@GLIBC_2.2.5
0000000000004010 D x
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P5_linker$ ./programB
x = 0x0 y = 0x3b6c
```

Figure 9: Linker- Part-B.

Program C Analysis

- In Module 1, the declaration `extern void foo(void)` introduces a reference to a global function symbol `foo` and requires that a corresponding global definition be provided by some module.
- In Module 2, `foo` is defined as `static`, giving it local linkage. Consequently, this definition is not visible outside Module 2 and cannot satisfy external references.
- As a result, the linker is unable to resolve the reference to `foo` from Module 1, leading to a link-time error.

Answers

- (a) REF($foo.1$) $\rightarrow$ ERROR
- (b) REF($x.1$) $\rightarrow$ DEF($x.2$)
- (c) REF($x.2$) $\rightarrow$ DEF($x.2$)

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P9$ gcc -c c_m1.c -o c_m1.o
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P9$ gcc -c c_m2.c -o c_m2.o
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P9$ nm -g c_m1.o
0000000000000000 T bar
                 U foo
                 U _GLOBAL_OFFSET_TABLE_
0000000000000004 C x
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P9$ nm -g c_m2.o
0000000000000000 D x
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P9$ gcc c_m1.o c_m2.o -o programC
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/9/../../../x86_64-linux-gnu/Scrt1.o: in function `_start':
(.text+0x24): undefined reference to `main'
/usr/bin/ld: c_m1.o: in function `bar':
c_m1.c:(.text+0x9): undefined reference to `foo'
collect2: error: ld returned 1 exit status
```

Figure 10: Linker- Part-C.

Problem 7: Heap vs Stack Growth Analysis

A process has two dynamically changing memory regions: the **heap** and the **stack**. In this problem, you will explore how these regions are laid out in **virtual memory** and how they grow during program execution.

Task

Write a C program that:

- Allocates multiple heap objects using `malloc()` and `calloc()`
- Uses recursion to generate multiple stack frames
- Prints the memory addresses of:
  - heap allocations
  - a local variable inside each recursive call

Constraints

Your program must satisfy the following:

- Allocate **at least three heap blocks**
- Use `realloc()` on **at least one heap block**
- Use a recursion depth of ≥ 5
- Print memory addresses at every step

Skeleton Code

You are provided with a skeleton. You must complete the missing parts.

```
#include <stdio.h>
#include <stdlib.h>

void recurse(int depth) {
    int stack_var;
    // TODO: Print address of stack_var

    if (depth > 0)
        recurse(depth - 1);
```

```
}

int main() {
    // TODO: Allocate memory using malloc
    // TODO: Allocate memory using calloc
    // TODO: Reallocate one block using realloc

    // TODO: Print heap addresses

    recurse(5);
    /* ---- DO NOT REMOVE THIS LINE ---- */
    getchar();   // pause execution for memory inspection

    // TODO: Free all allocated memory
    return 0;
}
```

Questions

1. Compile the program and inspect the type and content of the generated file. What segments of code are visible at this stage?
2. Based on runtime output:
   (a) Does the stack grow toward higher or lower memory addresses?
   (b) Does the heap grow toward higher or lower memory addresses?
3. Does `realloc()` always preserve the original memory address? Justify your answer using observations.
4. Do memory addresses remain the same across multiple program runs? Explain the reason for any variation.
5. Perform runtime memory analysis of the program and determine the address ranges corresponding to the heap and stack segments.
6. Does memory growth always occur in page-sized units? Answer True or False and justify your answer using valid runtime outputs.

Solution

Complete Program

```
#include <stdio.h>
```

```
#include <stdlib.h>

void recurse(int depth) {
    int stack_var;
    printf("Stack var at depth %d: %p\n", depth, &stack_var);
    if (depth > 0)
        recurse(depth - 1);
}

int main() {
    int *h1 = malloc(16);
    int *h2 = malloc(16);
    int *h3 = calloc(4, sizeof(int));

    printf("Heap h1: %p\n", h1);
    printf("Heap h2: %p\n", h2);
    printf("Heap h3: %p\n", h3);

    h2 = realloc(h2, 64);
    printf("Heap h2 after realloc: %p\n", h2);

    printf("\nStack growth:\n");
    recurse(5);

    getchar();

    free(h1);
    free(h2);
    free(h3);
    return 0;
}
```

Execution Steps and Analysis

Step 1: Compilation

```
gcc -O0 -g heap_stack.c -o heap_stack
```

Explanation:

- `-O0` disables optimizations and preserves stack frames

- `-g` includes debugging information

Step 2: Inspect Generated File

```
file heap_stack
```

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4$ gcc -Wall -O0 -g heap_stack.c -o heap_stack
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4$ file heap_stack
heap_stack: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x8
6-64.so.2, BuildID[sha1]=10f86dceb048d21a77635baf2e62422a90802219, for GNU/Linux 3.2.0, with debug_info, not stripped
```

Figure 11: File type of the compiled executable

The output indicates an ELF 64-bit dynamically linked executable. At this stage, no heap or stack memory has been allocated.

Step 3: Inspect Executable Sections

```
readelf -S heap_stack
```

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4$ readelf -S heap_stack
There are 36 section headers, starting at offset 0x4478:

Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  [ 0]                   NULL             0000000000000000  00000000
       0000000000000000  0000000000000000           0     0     0
  [ 1] .interp           PROGBITS         0000000000000318  00000318
       000000000000001c  0000000000000000   A       0     0     1
  [ 2] .note.gnu.propert NOTE             0000000000000338  00000338
       0000000000000020  0000000000000000   A       0     0     8
  [ 3] .note.gnu.build-i NOTE             0000000000000358  00000358
       0000000000000024  0000000000000000   A       0     0     4
  [ 4] .note.ABI-tag     NOTE             000000000000037c  0000037c
       0000000000000020  0000000000000000   A       0     0     4
  [ 5] .gnu.hash         GNU_HASH         00000000000003a0  000003a0
       0000000000000024  0000000000000000   A       6     0     8
  [ 6] .dynsym           DYNSYM           00000000000003c8  000003c8
       0000000000000150  0000000000000018   A       7     1     8
  [ 7] .dynstr           STRTAB           0000000000000518  00000518
       00000000000000c7  0000000000000000   A       0     0     1
  [ 8] .gnu.version      VERSYM           00000000000005e0  000005e0
       000000000000001c  0000000000000002   A       6     0     2
  [ 9] .gnu.version_r    VERNEED          0000000000000600  00000600
       0000000000000030  0000000000000000   A       7     1     8
  [10] .rela.dyn         RELA             0000000000000630  00000630
       00000000000000c0  0000000000000018   A       6     0     8
  [11] .rela.plt         RELA             00000000000006f0  000006f0
       00000000000000c0  0000000000000018  AI       6    24     8
  [12] .init             PROGBITS         0000000000001000  00001000
       000000000000001b  0000000000000000  AX       0     0     4
```

Figure 12: ELF sections visible before execution

The following sections are visible:

- `.text` — program instructions
- `.data` — initialized global variables
- `.bss` — uninitialized global variables
- `.rodata` — read-only constants
- `.symtab` — symbol table (present due to `-g`)

Note: Heap and stack are not fixed sections in the executable. They are created dynamically at runtime.

Step 4: Runtime Execution

```
./heap_stack
```

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4$ ./heap_stack
Heap h1: 0x55cb683522a0
Heap h2: 0x55cb683522c0
Heap h3: 0x55cb683522e0
Heap h2 after realloc: 0x55cb68352710

Stack growth:
Stack var at depth 5: 0x7ffd321bdec4
Stack var at depth 4: 0x7ffd321bde94
Stack var at depth 3: 0x7ffd321bde64
Stack var at depth 2: 0x7ffd321bde34
Stack var at depth 1: 0x7ffd321bde04
Stack var at depth 0: 0x7ffd321bddd4

sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4$ ./heap_stack
Heap h1: 0x55eeb85102a0
Heap h2: 0x55eeb85102c0
Heap h3: 0x55eeb85102e0
Heap h2 after realloc: 0x55eeb8510710

Stack growth:
Stack var at depth 5: 0x7ffffb0f2a04
Stack var at depth 4: 0x7ffffb0f29d4
Stack var at depth 3: 0x7ffffb0f29a4
Stack var at depth 2: 0x7ffffb0f2974
Stack var at depth 1: 0x7ffffb0f2944
Stack var at depth 0: 0x7ffffb0f2914
```

Figure 13: Runtime heap and stack address output

Observations

- Heap allocation addresses increase with successive allocations
- Stack variable addresses decrease with deeper recursion

- `realloc()` may return a different address
- Memory addresses vary across multiple executions

Conclusions

- The stack grows toward **lower memory addresses**
- The heap grows toward **higher memory addresses**
- `realloc()` does not guarantee address preservation
- Address variation across runs is due to **Address Space Layout Randomization (ASLR)**

Step 5: Observing the Memory Map of a Running Process

To inspect the virtual memory layout of the running program, the execution is intentionally paused by inserting a `getchar()` call in the source code. This allows the process to remain active while its memory map is examined.

Commands

In a separate terminal, identify the process ID (PID):

```
ps aux | grep heap_stack
```

After identifying the PID, inspect the process memory map using:

```
cat /proc/<PID>/maps
```

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4$ ps aux | grep heap_stack
sonali   1063810  0.0  0.0   2496   580 pts/0    S+   19:09   0:00 ./heap_stack
sonali   1063812  0.0  0.0   9040  2360 pts/1    S+   19:09   0:00 grep --color=auto heap_stack
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4$ cat /proc/1063810/maps
55778a63d000-55778a63e000 r--p 00000000 103:06 11051141                  /home/sonali/Documents/IIT_GN/sem_4/COA_TA/T4/heap_stack
55778a63e000-55778a63f000 r-xp 00001000 103:06 11051141                  /home/sonali/Documents/IIT_GN/sem_4/COA_TA/T4/heap_stack
55778a63f000-55778a640000 r--p 00002000 103:06 11051141                  /home/sonali/Documents/IIT_GN/sem_4/COA_TA/T4/heap_stack
55778a640000-55778a641000 r--p 00002000 103:06 11051141                  /home/sonali/Documents/IIT_GN/sem_4/COA_TA/T4/heap_stack
55778a641000-55778a642000 rw-p 00003000 103:06 11051141                  /home/sonali/Documents/IIT_GN/sem_4/COA_TA/T4/heap_stack
5577c2430000-5577c2451000 rw-p 00000000 00:00 0                          [heap]
7ff5cdfab000-7ff5cdfcd000 r--p 00000000 103:05 822188                    /usr/lib/x86_64-linux-gnu/libc-2.31.so
7ff5cdfcd000-7ff5ce145000 r-xp 00022000 103:05 822188                    /usr/lib/x86_64-linux-gnu/libc-2.31.so
7ff5ce145000-7ff5ce193000 r--p 0019a000 103:05 822188                    /usr/lib/x86_64-linux-gnu/libc-2.31.so
7ff5ce193000-7ff5ce197000 r--p 001e7000 103:05 822188                    /usr/lib/x86_64-linux-gnu/libc-2.31.so
7ff5ce197000-7ff5ce199000 rw-p 001eb000 103:05 822188                    /usr/lib/x86_64-linux-gnu/libc-2.31.so
7ff5ce199000-7ff5ce19f000 rw-p 00000000 00:00 0
7ff5ce1e1000-7ff5ce1e2000 r--p 00000000 103:05 822184                    /usr/lib/x86_64-linux-gnu/ld-2.31.so
7ff5ce1e2000-7ff5ce205000 r-xp 00001000 103:05 822184                    /usr/lib/x86_64-linux-gnu/ld-2.31.so
7ff5ce205000-7ff5ce20d000 r--p 00024000 103:05 822184                    /usr/lib/x86_64-linux-gnu/ld-2.31.so
7ff5ce20e000-7ff5ce20f000 r--p 0002c000 103:05 822184                    /usr/lib/x86_64-linux-gnu/ld-2.31.so
7ff5ce20f000-7ff5ce210000 rw-p 0002d000 103:05 822184                    /usr/lib/x86_64-linux-gnu/ld-2.31.so
7ff5ce210000-7ff5ce211000 rw-p 00000000 00:00 0
7fff447a8000-7fff447c9000 rw-p 00000000 00:00 0                          [stack]
7fff447cd000-7fff447d1000 r--p 00000000 00:00 0                          [vvar]
7fff447d1000-7fff447d3000 r-xp 00000000 00:00 0                          [vdso]
ffffffffff600000-ffffffffff601000 --xp 00000000 00:00 0                  [vsyscall]
```

Figure 14: Runtime virtual memory map showing heap and stack address ranges.

From the runtime memory map, the address ranges corresponding to the stack and heap segments can be identified as follows:

- **Stack:** `7fff447a8000--7fff447c9000`
- **Heap:** `5577c2430000--5577c2451000`

Step 6: Memory Growth and Page-Sized Allocation

Answer: True.
Memory growth occurs in page-sized units because virtual memory management is performed at page granularity by the operating system. The smallest unit that can be mapped, protected, or allocated is a memory page, which is typically 4 KB on x86-64 systems.

Command

To observe memory region sizes at runtime, execute:

```
pmap <PID>
```

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4$ pmap 1063810
1063810:   ./heap_stack
000055778a63d000      4K r---- heap_stack
000055778a63e000      4K r-x-- heap_stack
000055778a63f000      4K r---- heap_stack
000055778a640000      4K r---- heap_stack
000055778a641000      4K rw--- heap_stack
00005577c2430000    132K rw---   [ anon ]
00007ff5cdfab000    136K r---- libc-2.31.so
00007ff5cdfcd000   1504K r-x-- libc-2.31.so
00007ff5ce145000    312K r---- libc-2.31.so
00007ff5ce193000     16K r---- libc-2.31.so
00007ff5ce197000      8K rw--- libc-2.31.so
00007ff5ce199000     24K rw---   [ anon ]
00007ff5ce1e1000      4K r---- ld-2.31.so
00007ff5ce1e2000    140K r-x-- ld-2.31.so
00007ff5ce205000     32K r---- ld-2.31.so
00007ff5ce20e000      4K r---- ld-2.31.so
00007ff5ce20f000      4K rw--- ld-2.31.so
00007ff5ce210000      4K rw---   [ anon ]
00007fff447a8000    132K rw---   [ stack ]
00007fff447cd000     16K r----   [ anon ]
00007fff447d1000      8K r-x--   [ anon ]
ffffffffff600000      4K --x--   [ anon ]
 total             2500K
```

Figure 15: Memory regions allocated in page-sized units.

Problem 8: Symbol Table - I

Starter Code

```
#include <stdio.h>
#include <stdlib.h>
```

```
int global_var = 100;
static int static_global = 200;

void public_func() {}
static void private_func() {}

void recurse(int depth) {
    int stack_var;
    if (depth > 0)
        recurse(depth - 1);
}

int main() {
    int *h1 = malloc(16);
    int *h3 = calloc(4, sizeof(int));
    printf("%d\n", global_var);
    recurse(3);
    getchar();
    return 0;
}
```

Questions

(a) Which symbols are marked with `T`, `t`, `U`, `D`, `d`, and `B`?

(b) Which symbols are global and which are local? What is the effect of `static`?

(c) Will `private_func` be visible outside this object file?

(d) Are symbol addresses preserved?

(e) Do `h1`, `h3`, and `stack_var` appear in the symbol table?

Solution

(a)

Symbol	Meaning	Section
T	Global function	.text
t	Static function	.text
U	Undefined	External
D	Global initialized	.data
d	Static initialized	.data
B	Global uninitialized	.bss
b	Static uninitialized	.bss

(b) Global: `main`, `public_func`, `global_var`. Local: `private_func`, `static_global`. `static` restricts linkage to one translation unit.

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P6$ nm symbol_table | grep "private_func\|static_global"
00000000000011b4 t private_func
0000000000004014 d static_global
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P6$ nm symbol_table | grep "main\|public_func\|global_var"
0000000000004010 D global_var
                 U __libc_start_main@@GLIBC_2.2.5
00000000000011e4 T main
00000000000011a9 T public_func
```

Figure 16: Symbol table showing global and local symbol visibility.

(c) False. The compiler restricts visibility and appears only in the object file, but other object files cannot link to it.

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P6$ nm symbol_table | grep private_func
00000000000011b4 t private_func
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P6$
```

Figure 17: Local text symbol `private_func` marked with `t`, indicating internal linkage and object-file scope.

(d) Addresses in object files are relative/virtual offsets. Linker resolves addresses when creating the final executable, hence addresses may change. Symbols linked statically get actual addresses in .text / .data.

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P6$ nm symbol_table.o | grep "recurse"
0000000000000016 T recurse
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P6$ nm symbol | grep "recurse"
00000000000011bf T recurse
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P6$
```

Figure 18: Symbol address relocation between object file and final executable after linking.

(e) No. Heap and stack variables are runtime entities and not part of the symbol table.

```
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P6$ nm symbol_table | grep -E "main|recurse|global_var|static_global|private_func
"
0000000000004010 D global_var
                 U __libc_start_main@@GLIBC_2.2.5
00000000000011e4 T main
00000000000011b4 t private_func
00000000000011bf T recurse
0000000000004014 d static_global
sonali@dhansiri:~/Documents/IIT_GN/sem_4/COA_TA/T4/P6$
```

Figure 19: Absence of heap and stack variables in the symbol table, confirming they are runtime-only entities.

Problem 9: Symbol Table - II

This problem concerns the `swap.o` module shown in Figure 7.1(b). For each symbol that is defined or referenced in `swap.o`, indicate whether or not it will have a symbol table entry in the `.symtab` section of module `swap.o`. If so, indicate the module that defines the symbol, the symbol type (local, global, or extern), and the section (`.text`, `.data`, or `.bss`) it occupies in that module.

Code: `swap.c`

```
/* swap.c */
extern int buf[];

int *bufp0 = &buf[0];
int *bufp1;

void swap()
{
    int temp;

    bufp1 = &buf[1];
    temp = *bufp0;
    *bufp0 = *bufp1;
    *bufp1 = temp;
}
```

Solution

The purpose of this problem is to understand the relationship between C variables and functions and the symbols that appear in the linker symbol table. In particular, note that C local variables (such as `temp`) do not have entries in the symbol table.

Symbol	swap.o .symtab entry?	Symbol type	Module where defined	Section
buf	yes	extern	main.o	.data
bufp0	yes	global	swap.o	.data
bufp1	yes	global	swap.o	.bss
swap	yes	global	swap.o	.text
temp	no	—	—	—

Explanation

- buf is declared with extern, so it is referenced in swap.o but defined in another module (main.o).
- bufp0 and bufp1 are global variables defined in swap.o. Since bufp1 is uninitialized, it is placed in the .bss section.
- swap is a globally visible function, stored in the .text section.
- temp is a local (automatic) variable allocated on the stack and therefore does not appear in the symbol table.